AUTONOMOUS AI ENGINEERING MASTER SPECIFICATION & SYSTEM PROMPT

# VECTRABASE AI — MULTI-TENANT RAG KNOWLEDGE ENGINE & EMBEDDABLE VUE.JS WIDGET

Production Monorepo Blueprint, Cross-Agent Continuity Protocol, & Empirical Performance Mandate ( `vectrabase-ai.vercel.app` )

**Sole Creator & Principal Architect:** Karthik Kathari **GitHub:** github.com/Karthik-Kathari

**LinkedIn:** linkedin.com/in/karthikkathari

> **SYSTEM DIRECTIVE TO AUTONOMOUS CODING AGENT (ANTIGRAVITY / CLAUDE CODE / CODEX):**
>
> You are operating as a Principal Staff Full-Stack Architect executing the complete build of **VectraBase AI** ( `vectrabase-ai.vercel.app` ) for **Karthik Kathari**. You must build a complete, working, production-verified full-stack B2B SaaS monorepo that real users, developers, and recruiters can use immediately with their own PDFs, Markdown docs, or website URLs—never a static UI prototype or mock wrapper. Every architectural claim (1,000,000 indexed vector/audit records, sub-50ms cursor pagination, 60fps DOM virtualization, Redis semantic caching & rate limiting, JWT silent refresh at field 48 of a 50-field form, and real-time FastAPI + `pgvector` streaming RAG citations) must be backed by working code, automated seed generators, and live benchmark telemetry inside the UI.
>
> **STRICT AUTHORSHIP RULE:** Only **Karthik Kathari** ( `github.com/Karthik-Kathari` | `linkedin.com/in/karthikkathari` | `karthikmk.workspace@gmail.com` ) may appear anywhere in the codebase, UI footers, package manifests, license files, commit attributions, seed data, or documentation. Never insert any other person's name, fictional team names, or AI tool credits anywhere in the project.

## 1. Cross-Agent Continuity & Anti-Token-Limit Protocol (Mandatory Step 0)

To guarantee zero work loss if Antigravity reaches a token limit, context window boundary, or session timeout, you MUST initialize and maintain two stateful files in the root directory before writing any application code: `PROGRESS_TRACKER.md` and `ARCHITECTURE_DECISIONS.md` .

### 1.1 Rules for Real-Time State Synchronization

- **Atomic File Updates:** Immediately after completing any sub-task, file creation, database migration, or test suite, update `PROGRESS_TRACKER.md` in the repository root. Never batch updates at the end of a phase.
- **Status Markers:** Use strict status tags for every task: `[x] COMPLETED` , `[-] IN_PROGRESS` , or `[ ] PENDING` .

- **Checkpoint Metadata:** Under the active `[-] IN_PROGRESS` section in `PROGRESS_TRACKER.md`, always record: (1) Exact file currently being edited, (2) Last verified shell command or test output, (3) Next immediate code action required, and (4) Any environment variables or port bindings established.

### 1.2 Mandatory Template for Root `PROGRESS_TRACKER.md`

```
# VECTRABASE AI — LIVE BUILD & CONTINUITY TRACKER
- **Project Creator & Owner:** Karthik Kathari (github.com/Karthik-Kathari)
- **Target Production URL:** https://vectrabase-ai.vercel.app
- **Last Active Phase:** Phase [X] - [Sub-task Name]
- **Last Updated Timestamp:** [ISO-8601 Timestamp]
- **Current Build Health:** [PASSING / IN_PROGRESS / BLOCKED]

## EXACT RESUME POINT (FOR AGENT HANDOFF)
- **Currently Active Task:** [Task ID & Description]
- **Last Completed File:** [File path]
- **Next Immediate Action:** [Exact file to create/edit & function to implement]
- **Verification Command to Run First:** `pnpm turbo run typecheck test`

## MASTER EXECUTION CHECKLIST
- [ ] Phase 0: Monorepo Initialization & Progress Tracker Setup
- [ ] Phase 1: Shared Zod Schemas, Prisma PostgreSQL + pgvector Schema & 1M-Row
Seed Engine
- [ ] Phase 2: Node.js API Gateway, JWT Silent Refresh & Redis Semantic Cache /
Rate Limiter
- [ ] Phase 3: Python FastAPI RAG Ingestion Pipeline, Hybrid Vector Search &
Streaming SSE
- [ ] Phase 4: Next.js 15 Enterprise Control Plane, GSAP Engine & 1M Virtual Chunk
Explorer
- [ ] Phase 5: 50-Field Enterprise AI Governance Wizard & Field-48 JWT Expiry
Simulator
- [ ] Phase 6: Vue.js 3 Embeddable AI Support Widget, Citation Highlighter & Live
Playground
- [ ] Phase 7: Automated Load Benchmarks (Autocannon), E2E Tests & Free Cloud
Deployment
```

### 1.3 Emergency Handoff Prompt (Copy-Paste to Claude Code / Codex / Cursor)

If Antigravity stops mid-build due to token exhaustion, Karthik Kathari will paste this exact one-line command into the replacement AI coding agent:

```
"Read the VectraBase AI Master Build Specification PDF and the root
PROGRESS_TRACKER.md file. Verify all [x] COMPLETED tasks by checking the
filesystem, do not rewrite completed working modules, and resume execution
immediately from the exact [-] IN_PROGRESS task while continuing to update
PROGRESS_TRACKER.md after every step. Ensure only Karthik Kathari appears as
creator."
```

## 2. Complete Project Overview & Real-World Problem Statement

**What VectraBase AI Is:** VectraBase AI ( `vectrabase-ai.vercel.app` ) is a production-grade, multi-tenant B2B Retrieval-Augmented Generation (RAG) Knowledge Engine and Embeddable AI Support Platform. It empowers any user, startup, or recruiter to upload real PDFs, technical documentation, or website URLs to instantly train a zero-hallucination AI knowledge base managed from a **Next.js 15** administrative control plane and deployed anywhere via a lightweight, customizable **Vue.js 3** embeddable chat & citation widget.

**Real-World Problem Solved:** Companies and developers want AI assistants trained on their own private documents, but existing solutions suffer from three critical engineering flaws: (1) LLM hallucinations without verifiable page/paragraph citations, (2) runaway API token costs caused by repeating identical user queries without semantic caching or rate limiting, and (3) bloated admin interfaces that crash the browser DOM when inspecting hundreds of thousands of indexed document chunks and query logs, or lose complex configuration state when short-lived JWT tokens expire.

**Why Real Users & Recruiters Use It Immediately:** Any visitor—whether a founder, developer, or tech recruiter evaluating **Karthik Kathari**—can drop a real PDF (such as a company handbook, API spec, or even Karthik's own resume/architecture guide) into VectraBase AI, watch the **FastAPI + pgvector** pipeline chunk and embed it in real time, and immediately ask complex questions inside the live **Vue.js 3** widget with streaming answers and highlighted source citations.

## 3. Technology Requirements & Monorepo Architecture

Organize the repository as a clean **Turborepo ( `pnpm` workspaces)** monorepo. Every technology listed below is mandatory and must be genuinely integrated into the runtime pipeline.

| Workspace / Layer | Required Technology Stack | Architectural Responsibility |
|---|---|---|
| `apps/web-dashboard` | **Next.js 15 (App Router, SSR)**, React 19, TypeScript, Tailwind CSS, **GSAP 3**, TanStack Query v5, `@tanstack/react-virtual` , React Hook Form, Axios | Primary B2B SaaS Control Plane: Document ingestion studio, 1,000,000-row virtualized vector chunk & telemetry explorer, 50-field AI Governance Wizard, embed script generator, and live SQL/DOM benchmark HUD. |
| `apps/embed-widget` | **Vue.js 3 (Composition API,** `<script setup>` **)**, TypeScript, Pinia, Vite, Tailwind CSS, **GSAP 3** | Embeddable Client AI Assistant & Interactive Sandbox: Streaming Markdown responses, clickable source citation cards with PDF page/chunk preview, semantic cache hit badge, and threat/load simulator presets. |
| `apps/api-gateway` | **Node.js (v20+)**, Express.js, TypeScript, Prisma ORM, PostgreSQL, **Redis (ioredis / Upstash)**, Server-Sent Events (SSE) / Socket.io, JWT, Bcrypt | Core Orchestration Engine: Multi-tenant RBAC, short-lived JWT auth + HTTP-only refresh rotation, Redis sliding-window rate limiting, SHA-256 ingestion idempotency locks, and real-time telemetry streaming. |

| Workspace / Layer | Required Technology Stack | Architectural Responsibility |
|---|---|---|
| `apps/rag-engine` | **Python 3.11+, FastAPI**, Pydantic v2, Uvicorn, Asyncpg, **PostgreSQL** **`pgvector`**, PyMuPDF / BeautifulSoup4, Free-Tier LLM & Embeddings (Groq / Gemini API + local deterministic fallback) | Production RAG Microservice: Asynchronous PDF/URL parsing, recursive semantic chunking, vector embedding generation, hybrid search (HNSW cosine similarity + full-text keyword scoring), and citation verification. |
| `packages/shared-types` | **TypeScript 5, Zod** | Single source of truth for end-to-end validation schemas, API contracts, 50-field governance config types, RBAC role enums, and streaming citation payloads across Next.js, Vue.js, and Node.js. |
| `packages/benchmark-suite` | **Autocannon**, SQL `EXPLAIN ANALYZE` runner, Vitest, Playwright | Automated verification scripts that seed 1,000,000 indexed chunk/query audit rows, execute concurrency stress tests, and write real JSON benchmark reports rendered inside the dashboard UI. |

### 3.1 Target Monorepo Directory Structure

```
vectrabase-ai/
├── PROGRESS_TRACKER.md                # Real-time cross-agent state tracker
(Mandatory)
├── ARCHITECTURE_DECISIONS.md          # Engineering trade-offs & benchmark logs
├── docker-compose.yml                 # Local Postgres 16 (pgvector) + Redis 7 +
Services
├── turbo.json                         # Turborepo pipeline configuration
├── package.json                       # Root workspace config (Author: Karthik
Kathari)
├── apps/
│   ├── web-dashboard/                 # Next.js 15 SaaS Control Plane (Port 3000)
│   ├── embed-widget/                  # Vue.js 3 Embeddable AI Widget & Sandbox
(Port 5173)
│   ├── api-gateway/                   # Node.js + Express + Prisma + Redis (Port
4000)
│   └── rag-engine/                    # Python FastAPI RAG & pgvector Engine
(Port 8000)
└── packages/
    ├── shared-types/                  # Shared Zod schemas & TypeScript
interfaces
    └── benchmark-suite/               # 1M-row generator, SQL profiler & load
tester
```

## 4. Complete End-to-End Application Flow

VectraBase AI operates as a cohesive multi-tenant ecosystem where document training in Next.js and live user queries in the Vue.js 3 widget flow seamlessly through Node.js, Redis, FastAPI, and PostgreSQL:

1. **Step 1 — Document Ingestion & Idempotent Deduplication (Next.js → Node.js → FastAPI):** A user uploads a real PDF, Markdown file, or URL in `apps/web-dashboard`. The Node.js gateway computes a SHA-256 content hash and sets an atomic Redis idempotency lock (`SETNX`) so duplicate uploads or double-clicks never re-index or waste embedding tokens. FastAPI extracts text, splits it into overlapping semantic chunks (512 tokens, 64-token overlap), generates embeddings, and stores them in PostgreSQL `pgvector` with HNSW indexing while streaming live progress back to Next.js.
2. **Step 2 — End-User Query via Embeddable Widget (Vue.js 3 Client):** A visitor opens the `apps/embed-widget` (either embedded on an external site via a copy-paste snippet or side-by-side inside the Next.js Split-Screen Playground) and asks a natural-language question.
3. **Step 3 — Edge Rate Limiting & Redis Semantic Cache Check (Node.js + Redis):** The query hits `apps/api-gateway`. A Redis Lua script enforces per-tenant/IP sliding-window token rate limiting. Next, the gateway checks Redis for a normalized query/semantic cache hit. If cached, the verified response and citations return in **<2ms** with a green `REDIS CACHE HIT (0.9ms)` badge—saving 100% of LLM compute costs.
4. **Step 4 — Hybrid Vector Retrieval & Grounded Streaming (Python FastAPI + pgvector):** On a cache miss, FastAPI embeds the user query, runs a hybrid search in PostgreSQL combining `pgvector` cosine similarity (`<=>`) and PostgreSQL `tsvector` keyword ranking, retrieves the top-K chunks above the confidence threshold, and streams the grounded answer token-by-token via Server-Sent Events (SSE) alongside exact source chunk IDs, page numbers, and similarity scores.
5. **Step 5 — Real-Time Telemetry & GSAP Pipeline Visualization (Next.js 15 Dashboard):** The Next.js dashboard receives the live telemetry packet, animates the active RAG pipeline hops using GSAP, logs the retrieval latency and token savings into the 1,000,000-row audit ledger, and updates the analytics charts in real time.

## 5. Complete Feature Requirements & Empirical Proof Mandates

Every feature below must be implemented in full and accompanied by verifiable runtime proof. Never hardcode static strings claiming "1M rows supported"—build the actual benchmarking mechanics specified below.

### 5.1 Feature 1: The 1,000,000-Record Vector & Telemetry Explorer + Live Benchmark HUD

- **High-Speed Deterministic Seed Engine:** Build a Prisma/SQL batch ingestion script using PostgreSQL `COPY` or `generate_series(1, 1000000)` that populates **1,000,000 realistic indexed document chunk & RAG query telemetry records** in under 25 seconds locally (with configurable scaling for free cloud storage limits).
- **Keyset Cursor Pagination (Zero `OFFSET` Degradation):** Implement cursor pagination using a composite B-Tree index `CREATE INDEX idx_telemetry_created_id ON rag_telemetry_logs (created_at DESC, id DESC)`. Fetching deep historical pages must execute in under 15ms.
- **60fps Frontend Virtualization:** Render the 1M-row explorer in Next.js using `@tanstack/react-virtual` so scrolling through 1,000,000 records keeps the actual browser DOM node count strictly below 60 `<tr>` rows at all times.

- **300ms Debounced Search + Redis Query Cache:** Searching across 1,000,000 records by document source, query text, tenant workspace, or latency status uses a custom 300ms debounce hook ( `useDebounce` ) and caches filtered query pages in Redis for 60 seconds.
- **Mandatory "Prove It" Benchmark Drawer:** Add a prominent **"Verify 1M Scale & SQL Execution Plan"** button above the table. Clicking it runs a live backend `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` query on the database and displays in the UI: (a) Live PostgreSQL Row Count, (b) Actual SQL Execution Time in milliseconds, (c) Index Scan proof ( `Index Scan using idx_telemetry_created_id` ), (d) Redis Cache Hit vs. Postgres Miss latency comparison, and (e) a live browser `document.querySelectorAll('[data-virtual-row]').length` counter proving DOM virtualization.

### 5.2 Feature 2: 50-Field Enterprise AI Governance Wizard & Field-48 JWT Expiration Simulator

- **Real-World Context:** Enterprise teams deploying a multi-tenant RAG assistant must configure a rigorous 50-field AI Governance, Security & Deployment Manifest (chunking parameters, embedding dimensions, prompt-injection firewalls, PII redaction rules, CORS origins, webhook endpoints, and RBAC quotas).
- **50 Structured Validated Fields:** Group the 50 fields logically across 5 steps (Workspace Identity, Vector & Chunking Hyperparameters, Guardrails & PII Masking, Widget Branding & CORS Security, and Rate-Limit & Webhook Quotas) managed by **React Hook Form + Zod**.
- **Dual-Layer Zero-Data-Loss Architecture:**
  1. **Local Persistence:** Every field change is debounced (200ms) and synced automatically to `IndexedDB` / `localStorage` with a live status badge (*"Draft auto-saved locally 0.1s ago"*).
  2. **Axios Request Queue & Silent Refresh Interceptor:** Authentication uses a short-lived JWT Access Token (15 minutes) and an HTTP-Only Rotating Refresh Token. When form submission returns `401 Unauthorized` , an Axios response interceptor pauses outgoing requests in a Promise queue, calls `POST /api/auth/refresh` silently in the background, attaches the new JWT header, and replays the exact 50-field configuration payload without losing a single character or reloading the page.
- **Interactive Recruiter Test Trigger ("Simulate Expiry at Field 48"):** Include an interactive testing banner on the wizard page titled **"Live Resilience Test: Expire JWT at Field 48"**. Clicking this button auto-fills fields 1 through 47 with valid enterprise RAG parameters, focuses field 48, forcibly expires the active JWT access token to `0s` TTL, and invites the recruiter to type into fields 48–50 and click Save. A live Interceptor Telemetry Console on the right side of the screen visualizes each step in real time: `POST /api/governance [401 Token Expired] → Interceptor Queued 50-Field Payload → POST /api/auth/refresh [200 New JWT Issued] → Automatic Retry POST /api/governance [200 Saved — Zero Data Loss Verified]` .

### 5.3 Feature 3: Vue.js 3 Embeddable AI Widget & Interactive Split-Screen Playground

- **Standalone & Embeddable Architecture:** Built in `apps/embed-widget` with Vue 3 Composition API (`<script setup>`), Pinia state store, and Tailwind CSS. Provide a copy-paste HTML embed snippet generator in the Next.js dashboard AND a **"Split-Screen Live Lab"** where recruiters interact with the Vue.js 3 widget on the left while watching the Next.js 15 backend telemetry on the right.
- **Interactive Citations & Source Inspector:** Every AI answer displays clickable citation pills (e.g., `[Source 1: System_Architecture.pdf — Page 4, Score: 0.94]`). Clicking a pill slides open a source card showing the exact raw paragraph retrieved from PostgreSQL `pgvector`.
- **4 One-Click Recruiter Test Scenarios in the Vue Widget:**
  1. *Ask Grounded Document Question:* Queries the uploaded PDF (or pre-loaded VectraBase + Karthik Kathari Architecture Knowledge Base) and streams a cited answer.
  2. *Test Redis Semantic Cache Speed:* Fires the same question twice to demonstrate latency dropping from ~380ms (Vector + LLM) to **<2ms (Redis Cache Hit)**.
  3. *Simulate Prompt-Injection / Hallucination Attack:* Sends an out-of-scope or adversarial prompt to prove the FastAPI guardrail strictly refuses ungrounded claims when similarity falls below threshold.
  4. *Simulate Duplicate Webhook / Token Burst:* Fires 15 rapid concurrent requests from the widget to demonstrate Redis sliding-window rate limiting and `SETNX` idempotency deduplication.

### 5.4 Feature 4: Real-Time PDF/URL Ingestion & Zero-Key Fallback Mode

- **Upload Real Files Anytime:** Users can drag-and-drop any real PDF, `.md`, or `.txt` file (up to 10MB) or paste a public URL to create a live searchable collection in seconds.
- **Always-On Deterministic Embedding & Generation Fallback:** Support free-tier API keys (`GROQ_API_KEY` or `GEMINI_API_KEY`) for ultra-fast LLM streaming, AND engineer a built-in local deterministic TF-IDF/vector + extractive citation synthesis fallback in FastAPI so the live demo on `vectrabase-ai.vercel.app` works with 100% reliability even if external LLM API quotas are exhausted.

## 6. Professional UI/UX Design Direction (Strict Anti-Generic Mandate)

The user interface must look like it was crafted by a principal product design team at **Linear, Vercel, Supabase, or Raycast**. Generic AI-generated templates, cliché glowing robots, messy purple-pink gradients, or bloated card layouts are strictly forbidden.

| Design Dimension | Mandatory Developer-First SaaS Specification |
| --- | --- |
| **Color Architecture** | **Carbon Obsidian & Electric Indigo/Cyan Precision Palette:** Deep architectural dark canvas (`#090A0F` base, `#11131C` surface, `#1E2230` 1px crisp structural borders) paired with a clean high-contrast Light Mode toggle. Semantic status tokens: Emerald Cache Hit / Indexed (`#10B981`), Amber Rate-Limit Warning (`#F59E0B`), Rose Guardrail Blocked (`#F43F5E`), and Electric Cyan Vector Accent (`#06B6D4`). |

| Design Dimension | Mandatory Developer-First SaaS Specification |
| --- | --- |
| **Typography & Numerals** | Use geometric sans-serif ( `Inter` / `Geist` ) for UI labels and strict monospace ( `JetBrains Mono` / `Geist Mono` ) with `font-variant-numeric: tabular-nums` for all vector similarity scores, millisecond latencies, token counts, chunk IDs, and timestamps. |
| **Layout & Density** | Crisp 1px-border Bento & Swiss grid layout with a collapsible workspace switcher, `Cmd+K` / `Ctrl+K` command palette, split-screen RAG pipeline inspector, and interactive vector similarity heatbars. |
| **Branding & Attribution** | Display a custom geometric vector-prism SVG mark for **VectraBase AI**. In the sidebar footer, header profile badge, and documentation modal, display: **"Architected & Engineered by Karthik Kathari"** with direct icon links to his GitHub, LinkedIn, and Portfolio. |

## 7. GSAP Animation & Interactive Micro-Interactions Specification

Use **GSAP 3 (GreenSock Animation Platform)** across both the Next.js 15 dashboard and the Vue.js 3 widget to deliver hardware-accelerated 60fps engineering telemetry visualizations:

- **Live 5-Stage RAG Pipeline Visualizer (GSAP Timeline + SVG Motion):** At the top of the RAG Studio, build an interactive horizontal pipeline map ( `1. Vue Widget Query → 2. Node/Redis Cache & Rate Lock → 3. FastAPI Embedding → 4. Postgres pgvector HNSW Search → 5. Grounded SSE Stream` ). Whenever a user asks a question or uploads a PDF, animate a high-precision data pulse along the SVG path using GSAP, lighting up the exact millisecond latency spent at each stage (and short-circuiting at Stage 2 on a Redis Cache Hit!).
- **Vector Chunk Splitting & Cosine Similarity Bar Animation:** When a document is ingested or queried, smoothly animate the chunk cards and similarity score bars ( `0.00 → 0.94` ) using GSAP staggered timelines and numerical counter tweening.
- **JWT Interceptor State Morphing:** In the 50-Field AI Governance Wizard, animate the token TTL countdown ring and smoothly morph the interceptor status banner from `401 Expired` → `Silent Cookie Refresh...` → `200 Saved (Zero Data Loss)` using GSAP timelines.
- **Strict Performance Guardrail:** combined with React `React.memo` , `useMemo` , and `useCallback` optimizations, animate only GPU-composited properties ( `transform` , `opacity` ) and clean up all GSAP contexts on unmount.

## 8. Security, Scalability & Load Testing Requirements

- **Authentication & Multi-Tenant RBAC:** Implement 3 distinct roles enforced on both frontend routes and Express middleware: `WORKSPACE_OWNER` (full ingestion, 50-field governance mutation, API key management), `AI_ENGINEER` (document training, prompt tuning, benchmark runner), and `VIEWER_ANALYST` (read-only telemetry & playground testing).
- **API & RAG Hardening:** Enforce Helmet security headers, strict CORS allowlists for widget embedding, Zod payload validation, SHA-256 file deduplication, prompt-injection sanitization, and parameterized Prisma queries.

- **Automated Concurrency & Load Testing Suite:** Inside `packages/benchmark-suite`, create an executable Node.js `autocannon` load-testing script (`pnpm run benchmark:api`) that fires 500 concurrent connections against the Node.js API Gateway and Redis Semantic Cache endpoint for 15 seconds, automatically writing the throughput (Requests/sec, p50, p95, p99 latency) into a JSON artifact rendered inside the dashboard's **System Benchmarks** tab.

## 9. Step-by-Step Development Roadmap (Execution Phases)

Execute the build strictly in the 8 sequential phases below. Update `PROGRESS_TRACKER.md` after completing every numbered item.

| Phase | Deliverables, Implementation Tasks & Verification Criteria |
|---|---|
| **Phase 0** Setup & Tracker | Initialize Turborepo (`pnpm`), TypeScript configs, Docker Compose (PostgreSQL 16 with `pgvector` + Redis 7), `.env.example`, and create root `PROGRESS_TRACKER.md` and `ARCHITECTURE_DECISIONS.md` with Karthik Kathari as sole author. |
| **Phase 1** Data & 1M Seed | Build `packages/shared-types` (Zod schemas for Documents, Vector Chunks, 50-Field Governance Manifest, Auth, Telemetry). Configure Prisma schema with `pgvector` extension and composite B-Tree indexes (`created_at DESC, id DESC`). Build and verify the 1,000,000-row deterministic seed generator and SQL `EXPLAIN ANALYZE` benchmark endpoint. |
| **Phase 2** Node Gateway | Implement `apps/api-gateway`: JWT Access (15m) + HTTP-Only Refresh Token rotation, RBAC middleware, Redis sliding-window rate limiter, Redis SHA-256 ingestion idempotency lock, Redis semantic query cache, cursor-paginated 1M-row telemetry API, and SSE real-time broadcaster. |
| **Phase 3** FastAPI RAG Engine | Implement `apps/rag-engine` in Python FastAPI: PDF/Markdown/URL text extractor, semantic chunker, embedding generator (Groq/Gemini free API + deterministic local vector fallback), hybrid `pgvector` cosine similarity + keyword search, prompt-injection guardrail, and streaming SSE citation generator. Pre-seed a default knowledge base so the demo works out-of-the-box. |
| **Phase 4** Next.js Dashboard | Build `apps/web-dashboard` (Next.js 15 App Router): Obsidian developer UI shell, GSAP 5-stage live RAG pipeline visualizer, real-time document ingestion studio, 1,000,000-row virtualized data grid (`@tanstack/react-virtual`) with 300ms debounced search, and the interactive **SQL & DOM Benchmark Verification Modal**. |
| **Phase 5** 50-Field Governance | Build the 50-Field Enterprise AI Governance Wizard in Next.js: React Hook Form + Zod validation, `IndexedDB` / `localStorage` instant draft sync, Axios 401 request queue + silent token refresh interceptor, and the one-click "**Simulate JWT Expiry at Field 48**" interactive visualizer. |
| **Phase 6** Vue 3 AI Widget | Build `apps/embed-widget` in Vue.js 3 (Composition API + Pinia + GSAP): sleek embeddable AI chat & citation assistant, clickable source chunk viewer, 4 one-click recruiter test presets (RAG query, Redis cache speed test, guardrail test, rate-limit burst), copy-paste embed script modal, and Split-Screen Playground integration inside Next.js. |

| Phase | Deliverables, Implementation Tasks & Verification Criteria |
|---|---|
| **Phase 7**<br>Tests & Deploy | Write Vitest unit/integration tests and Playwright E2E flows. Run `autocannon` load benchmarks and commit results. Configure GitHub Actions CI/CD and zero-cost deployment manifests for Vercel ( `vectrabase-ai.vercel.app` ), Neon Serverless Postgres ( `pgvector` ), Upstash Redis, and Render (Node.js + FastAPI) with a scheduled cron keep-alive workflow. |

## 10. Testing, 100% Free Cloud Deployment & Portfolio Presentation

### 10.1 Zero-Cost (₹0/Month) Production Deployment Architecture

- **Frontend Hosting (Vercel Free Hobby Tier — `vectrabase-ai.vercel.app` ):** Deploy both `apps/web-dashboard` (Next.js 15) and `apps/embed-widget` (Vue 3 Vite) from the same GitHub monorepo with automatic HTTPS and edge caching.
- **Vector & Relational Database (Neon Serverless PostgreSQL Free Tier):** Provision a free Neon Postgres instance with `CREATE EXTENSION IF NOT EXISTS vector;` enabled. Neon never pauses after 7 days of inactivity and wakes in milliseconds whenever a recruiter opens the link.
- **Semantic Cache & Rate Limiter (Upstash Serverless Redis Free Tier):** Connect via standard `REDIS_URL` for zero-cost semantic query caching, rate limiting, and SHA-256 upload deduplication.
- **Backend Microservices (Render Free Tier + GitHub Actions Warm-Up):** Provide ready-to-deploy `render.yaml` Blueprint files and multi-stage `Dockerfile` s for `api-gateway` and `rag-engine` , paired with a `.github/workflows/keep-alive.yml` cron job that pings `/health` every 14 minutes during business hours so recruiters experience zero cold starts.

### 10.2 Recruiter-First Portfolio Presentation Features

- **One-Click "Launch Recruiter Sandbox" Button:** On the landing/login screen, provide a prominent primary button that logs the visitor in immediately as **Karthik Kathari (Principal AI Systems Architect — Sandbox Admin)** with pre-indexed technical documents (including a searchable VectraBase Architecture Whitepaper and Karthik Kathari's Engineering Profile) ready to query immediately.
- **Interactive 4-Step "Engineering Proof Switcher" in Top Header:** Allow recruiters and hiring managers to jump with a single click to: (1) *Live PDF Upload & Split-Screen Vue.js 3 RAG Playground*, (2) *1M-Row Virtualized Explorer + Live SQL `EXPLAIN ANALYZE` Proof*, (3) *50-Field Form & Field-48 JWT Expiration Zero-Data-Loss Test*, and (4) *Redis Semantic Cache (<2ms) & Concurrency Benchmark Report*.
- **Flagship GitHub `README.md` :** Generate an executive-grade root `README.md` authored exclusively by **Karthik Kathari** containing system architecture diagrams (Mermaid), real benchmark tables, local 1-command Docker setup ( `docker compose up --build` ), API documentation, and direct links to Karthik Kathari's LinkedIn, GitHub, and Portfolio.